A1 Agent Engine

Platform Guide

Comprehensive guide for platform operators and domain architects

Generated: May 15, 2026

Table of Contents

Admin Console

The Admin Console is the central management interface for platform operators. Located at **localhost:3001** (development), it provides comprehensive tools to manage infrastructure, configure LLM providers, monitor system health, and oversee multi-tenant deployments. Platform operators use this interface to ensure the platform runs smoothly, securely, and cost-effectively.

1. Dashboard

Purpose: Real-time overview of platform health and activity.

Key Metrics:

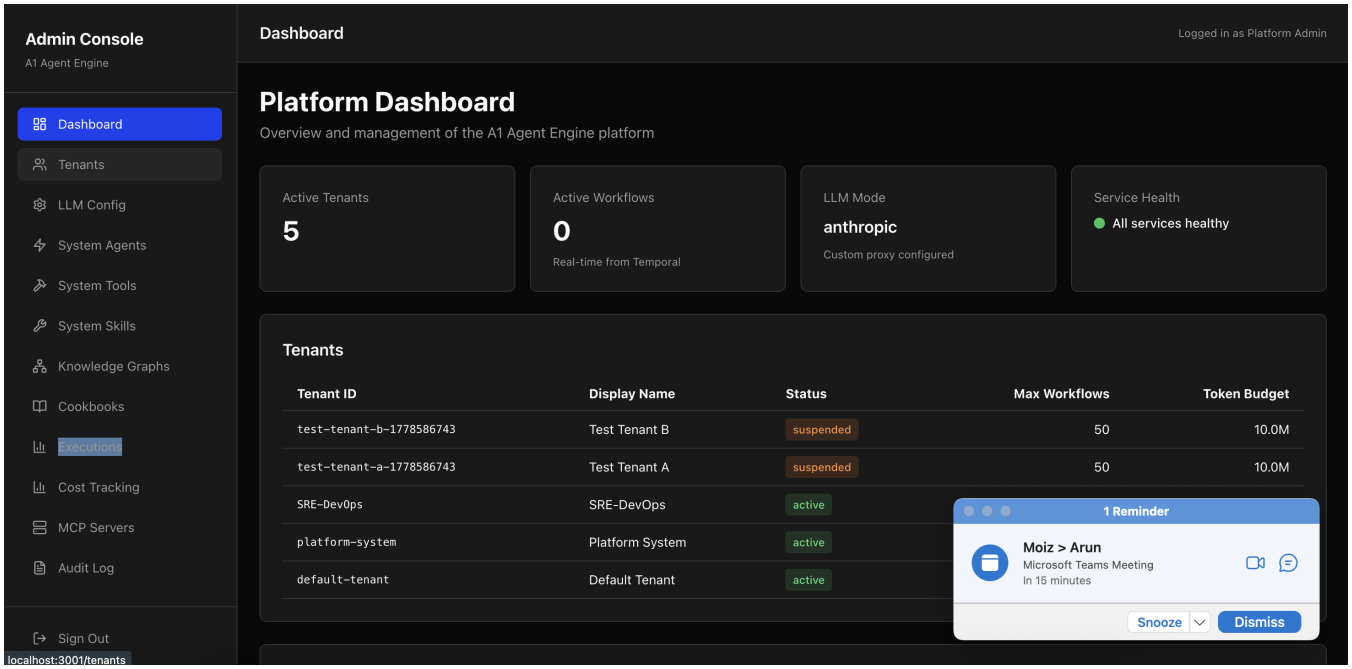
• Active Tenants: Number of tenant workspaces currently running

• Active Workflows: Real-time Temporal workflow count and status

• LLM Mode: Current LLM provider (Mock, Anthropic, OpenAI)

• Service Health: Status of all microservices and their endpoints

• Tenant Management: View all registered tenants with workflow quotas and token budgets



2. LLM Configuration

Purpose: Configure LLM providers and manage model access globally and per-tenant.

Platform LLM Mode: Select which LLM backend powers your agents:

• **Mock (Development):** Deterministic responses for testing and development without incurring costs

• **Anthropic:** Claude models via Anthropic API for production deployments

• **OpenAI:** GPT models via OpenAI API as an alternative provider

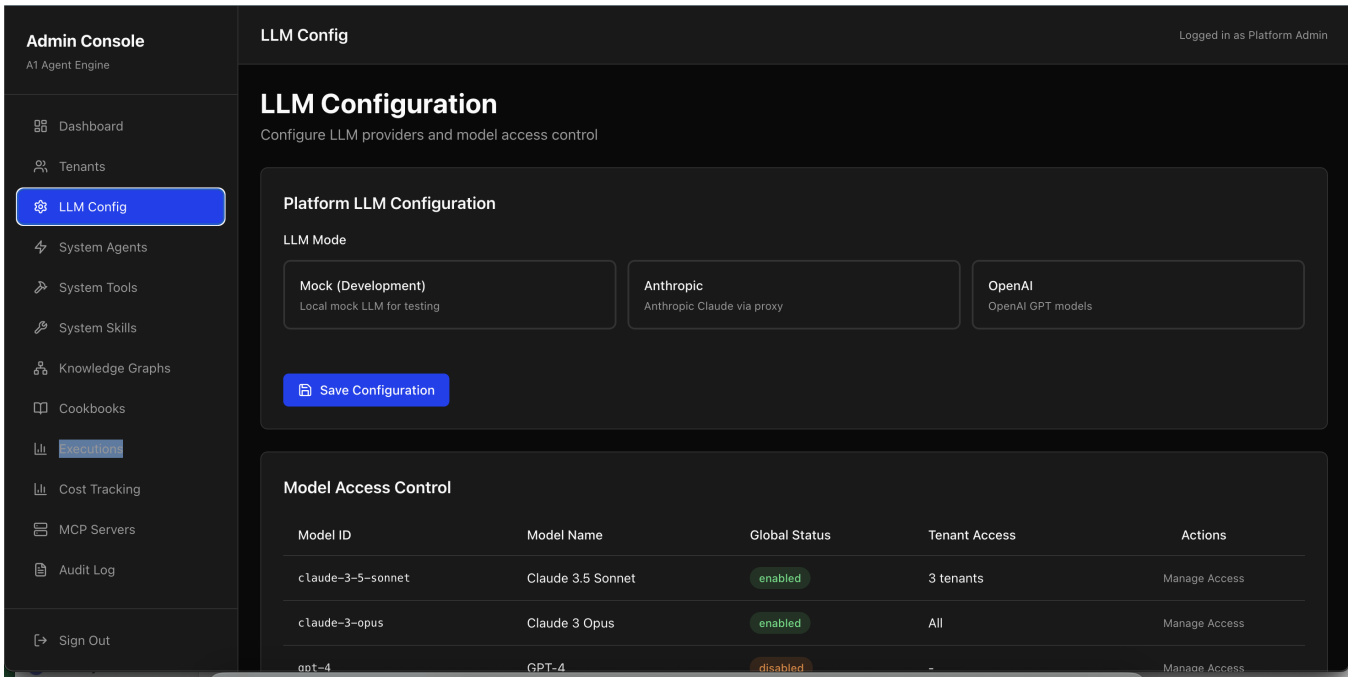
Model Access Control: Configure which models are available platform-wide or per-tenant:

• claude-3-5-sonnet - Recommended for balanced performance and cost

• claude-3-opus - Most capable model for complex reasoning tasks

• gpt-4 - OpenAI alternative for specific use cases

Per-Model Configuration: Enable or disable models globally or per-tenant, set API key routing, configure fallback models, and monitor token usage and costs in real-time.



3. System Tools

Purpose: Manage platform-level tools available to all agents across all tenants.

Tool Categories:

Infrastructure Tools:

• bash - Shell command execution (sandbox-controlled)

• deployment-checker - Kubernetes deployment validation

• log-analyzer - Log aggregation and analysis

Knowledge Graph Tools:

• kg-create-graph - Create knowledge graphs

• kg-add-node - Add nodes to KGs

• kg-add-edge - Create relationships

• kg-query - Query KGs

• kg-search - Full-text search over KGs

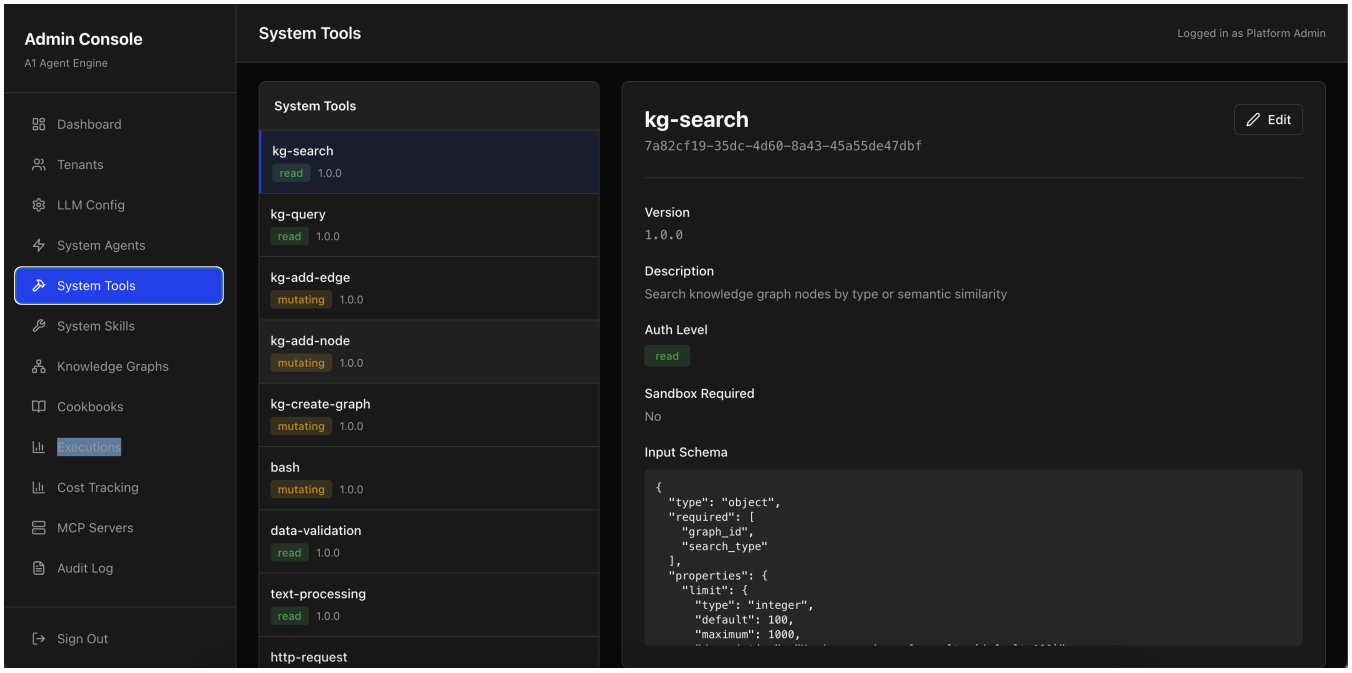
• kg-semantic-search - NLP-powered semantic search

Data Tools:

• http-request - HTTP API calls

• code-executor - Python/JavaScript execution (sandboxed)

Tool Configuration: View tool documentation, track usage across tenants, and monitor execution metrics.



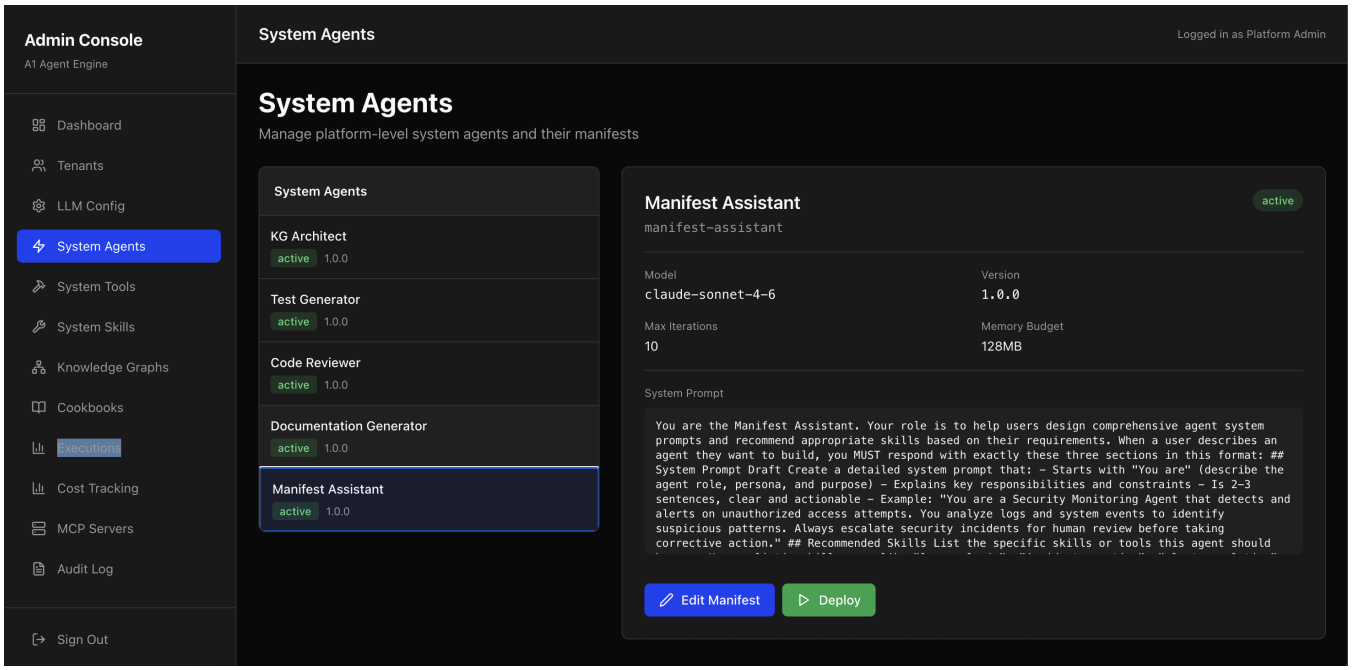
4. System Agents

Purpose: Pre-built system agents for platform operations and automation.

Available System Agents:

Agent	Version	Use Case
Manifest Assistant	1.0.0	Help with agent manifest generation
Test Generator	1.0.0	Generate unit tests for code changes
Code Reviewer	1.0.0	Peer code review automation
Deployment Checker	1.0.0	Validate deployment configurations

Features: View agent manifests, enable/disable per tenant, monitor execution logs, and manage agent versions.



5. Cookbooks

Purpose: Domain-specific agent templates for rapid deployment and knowledge sharing.

Cookbook Anatomy:

• **Overview:** Description and use case, domain tags, version and maintenance status

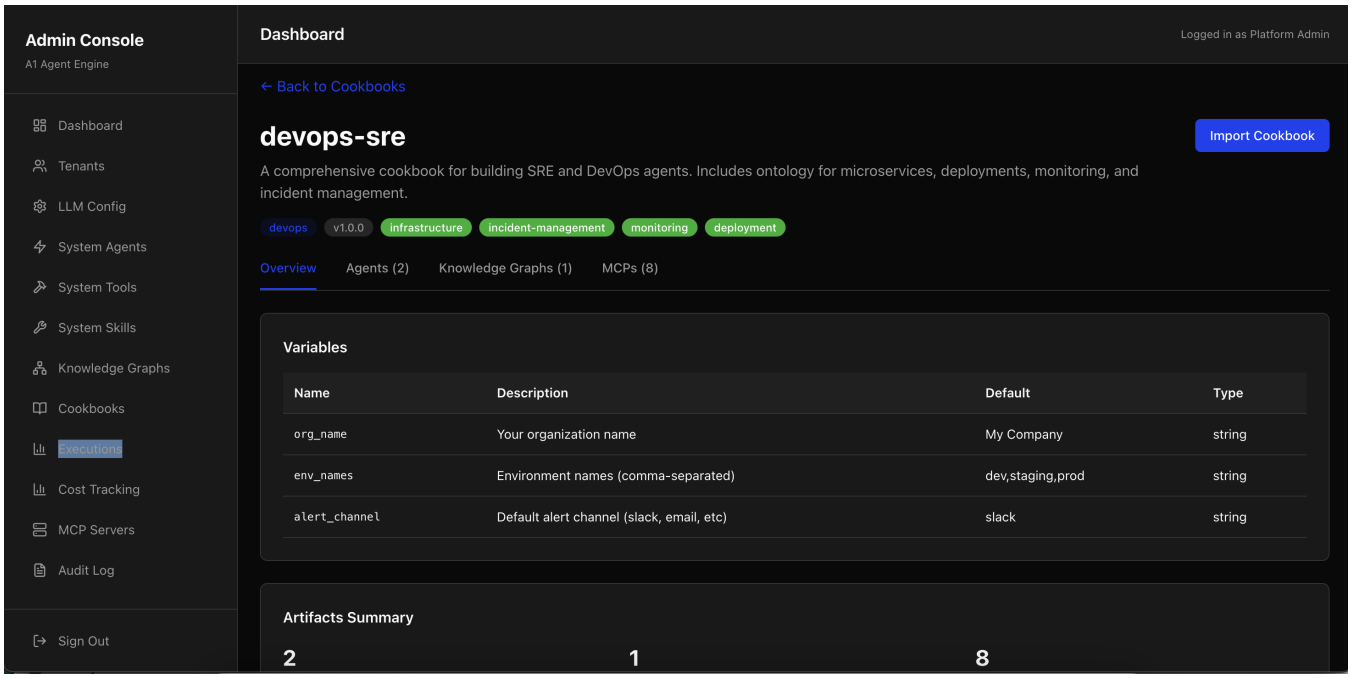
• **Variables:** Parameterized configuration (org_name, env_names, alert_channel, etc.)

• **Agents:** Pre-built agent definitions ready to import

• **Knowledge Graphs:** Domain ontologies and relationships included

• **MCP Recommendations:** Suggested MCP servers for the domain

Example: DevOps-SRE Cookbook includes 2 agents (deployment-validator, incident-responder), 1 Knowledge Graph (DevOps Platform ontology with 14 nodes and 19 relationships), and 8 MCP recommendations.



6. Knowledge Graphs

Purpose: Store and manage domain ontologies with interactive visualization and semantic search.

Knowledge Graph Features:

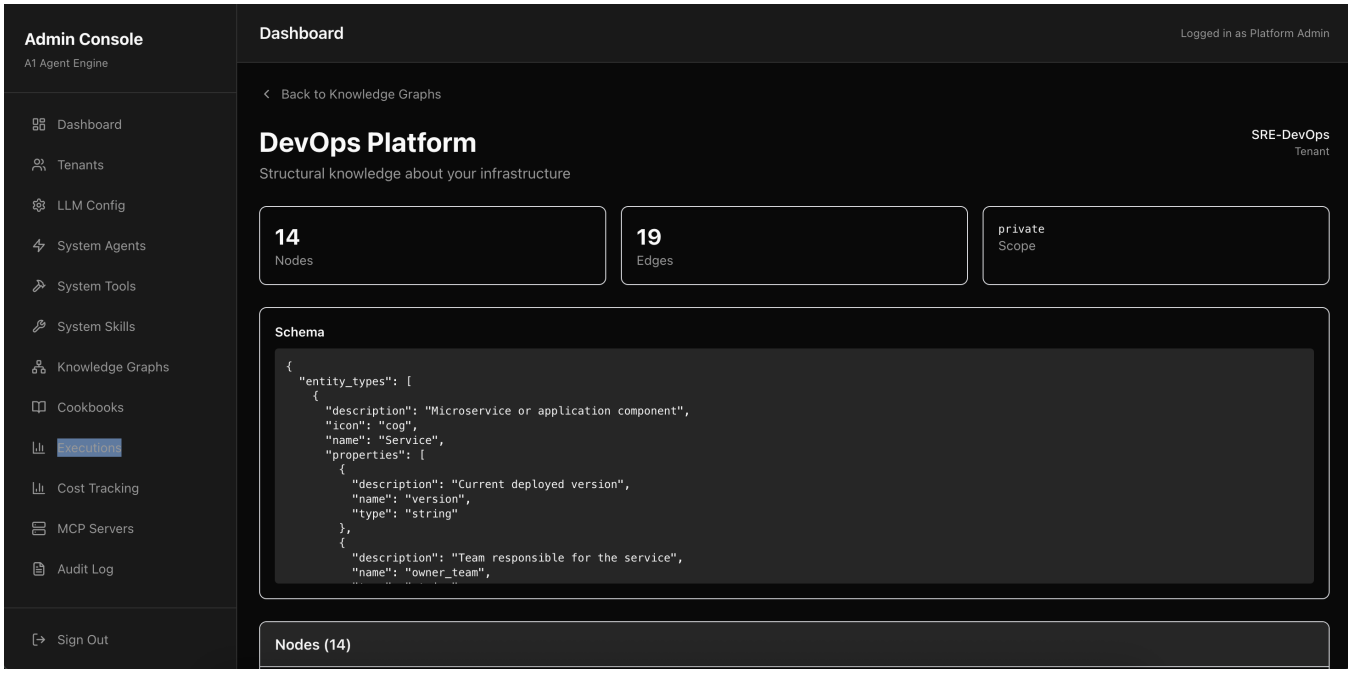
• **Structure:** Nodes (entities), Edges (relationships), Schema (type definitions), Embedding (auto-embedding for semantic search)

• **Visualization:** Interactive node-link diagram with color-coded nodes and relationship edges

• **Interaction:** Filter by node type, search across the graph, view node attributes, explore relationships

• **Capabilities:** Full-text search, semantic similarity search, relationship traversal, schema validation, version history

Use Cases: Infrastructure relationship mapping, service dependency graphs, team hierarchy and ownership, feature/capability trees, policy compliance mappings.



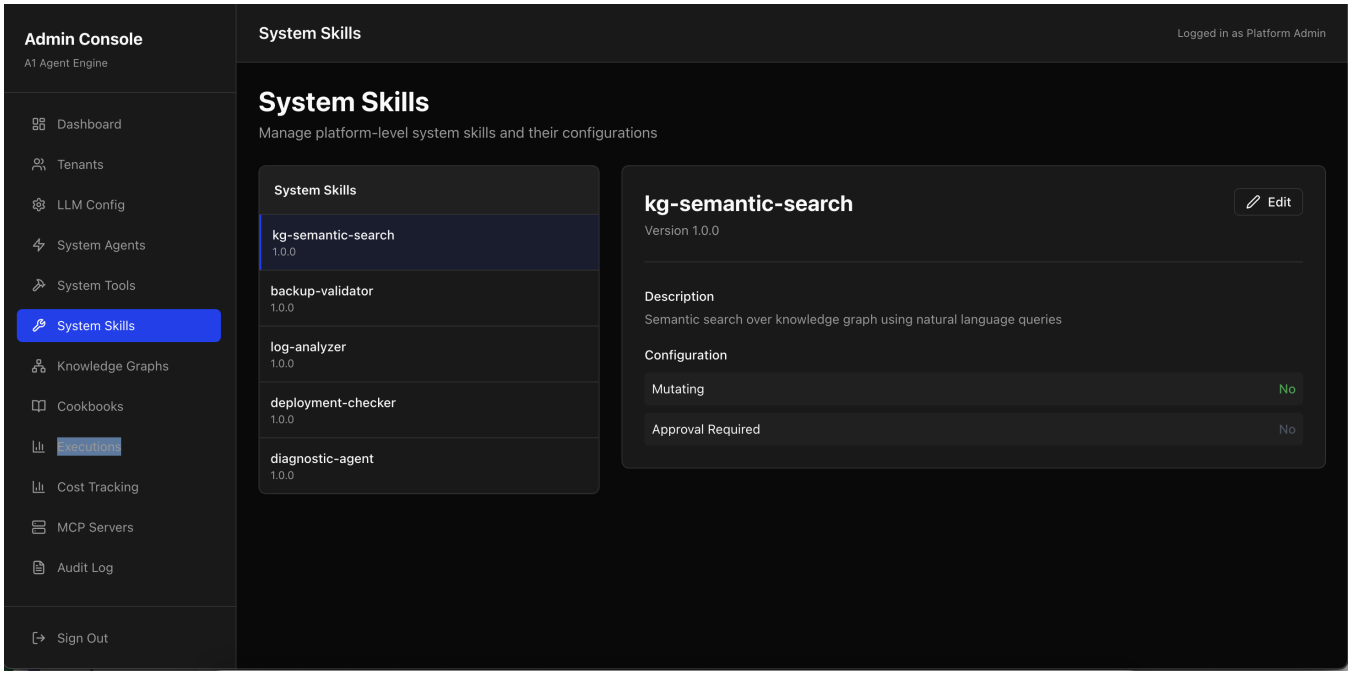
7. System Skills

Purpose: Pre-built domain skills for reuse across agents with approval controls.

Available System Skills:

Skill	Mutating	Requires Approval
kg-semantic-search	No	No
backup-validator	No	No
log-analyzer	No	No
deployment-checker	Yes	Yes
diagnostic-agent	Yes	Yes

Features: Browse skill catalog, view documentation, configure approval requirements (HITL), and monitor skill usage.



8. MCP Servers

Purpose: Manage Model Context Protocol integrations for extending platform capabilities.

MCP Server Management:

• Register global MCP servers available to all tenants

• Configure authentication (API keys, OAuth) for each server

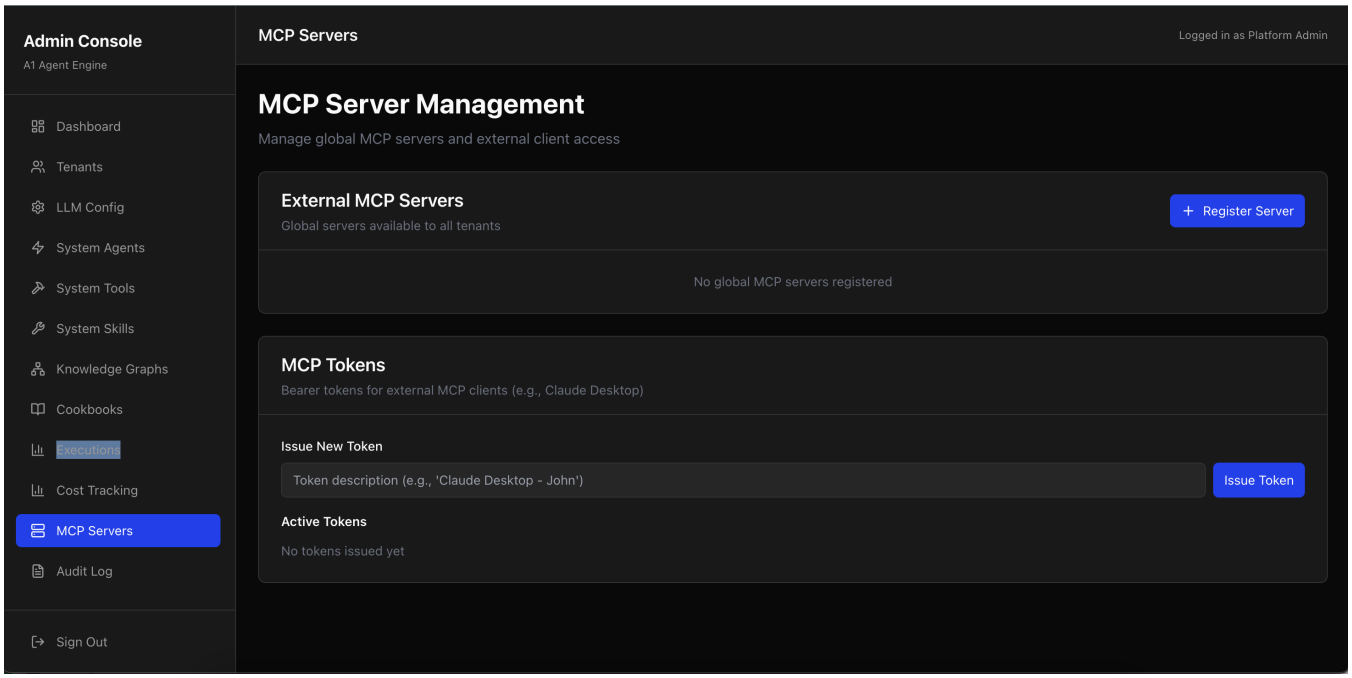
• Monitor server health and connectivity in real-time

• Issue tokens for external MCP clients (e.g., Claude Desktop)

• Manage token lifecycle and permissions

• Revoke tokens as needed for security

Benefits: Integrate external AI models, extend platform with custom tools via MCP protocol, and manage multi-agent ecosystems across different providers.



Agent Studio

Agent Studio is the workspace for domain architects to design, test, and manage agents. Located at **localhost:3000** (development), it provides tools to compose agents from skills, manage knowledge graphs, test agents in real-time, and access domain templates. This is where you build and validate the autonomous agents that solve your business problems.

1. Agents

Purpose: Create and manage autonomous agents with LLM models, skills composition, and custom system prompts.

Agent Definition:

• Name and description for identification and discovery

• Model selection (Claude, GPT) - the LLM backbone

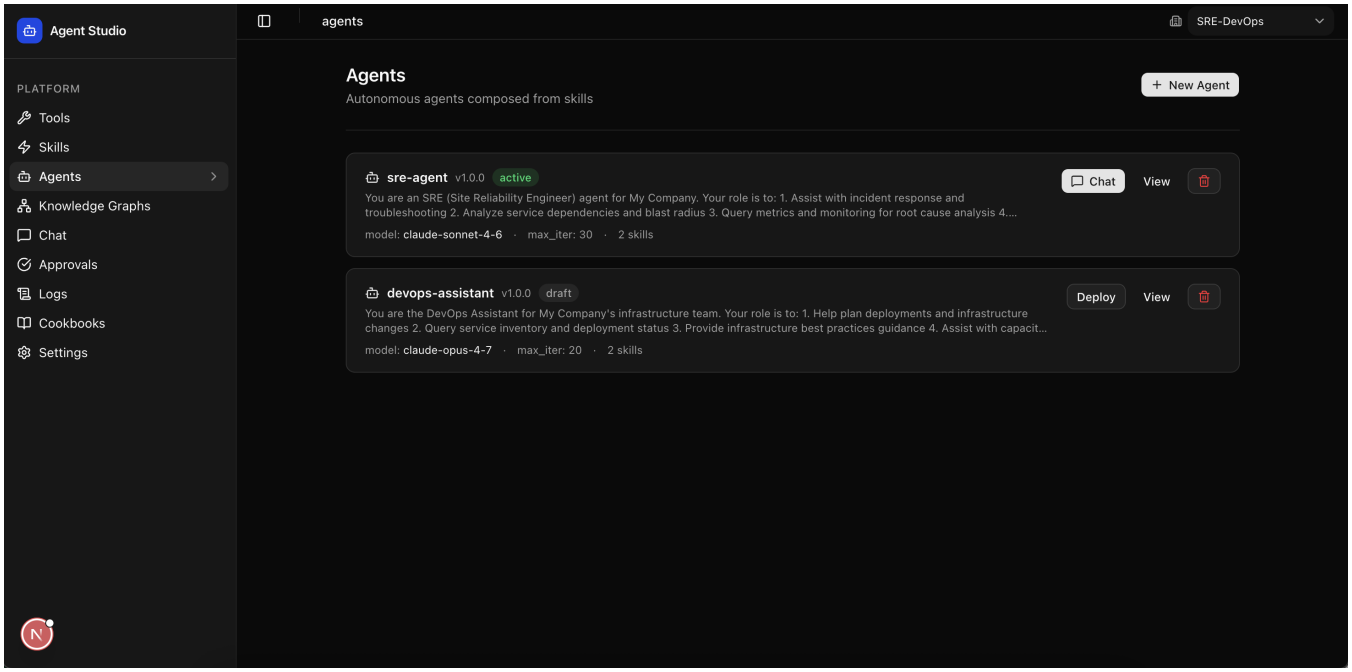
• Max iterations - control runaway agent behavior

• Skills composition - which domain capabilities to include

• System prompt customization - guide agent behavior and focus

• Tools access - read-only or full capability access

Agent Lifecycle: Draft (in development) → Active (ready for use) → Deprecated (no longer recommended) → Archived (legacy agents). Each agent executes as a Temporal durable workflow, surviving failures and maintaining an audit trail.



2. Knowledge Graphs

Purpose: Browse and visualize domain knowledge to understand the context available to your agents.

KG Interaction:

• Visual node-link diagram showing entities and their relationships

• Filter by node type to focus on specific entity categories

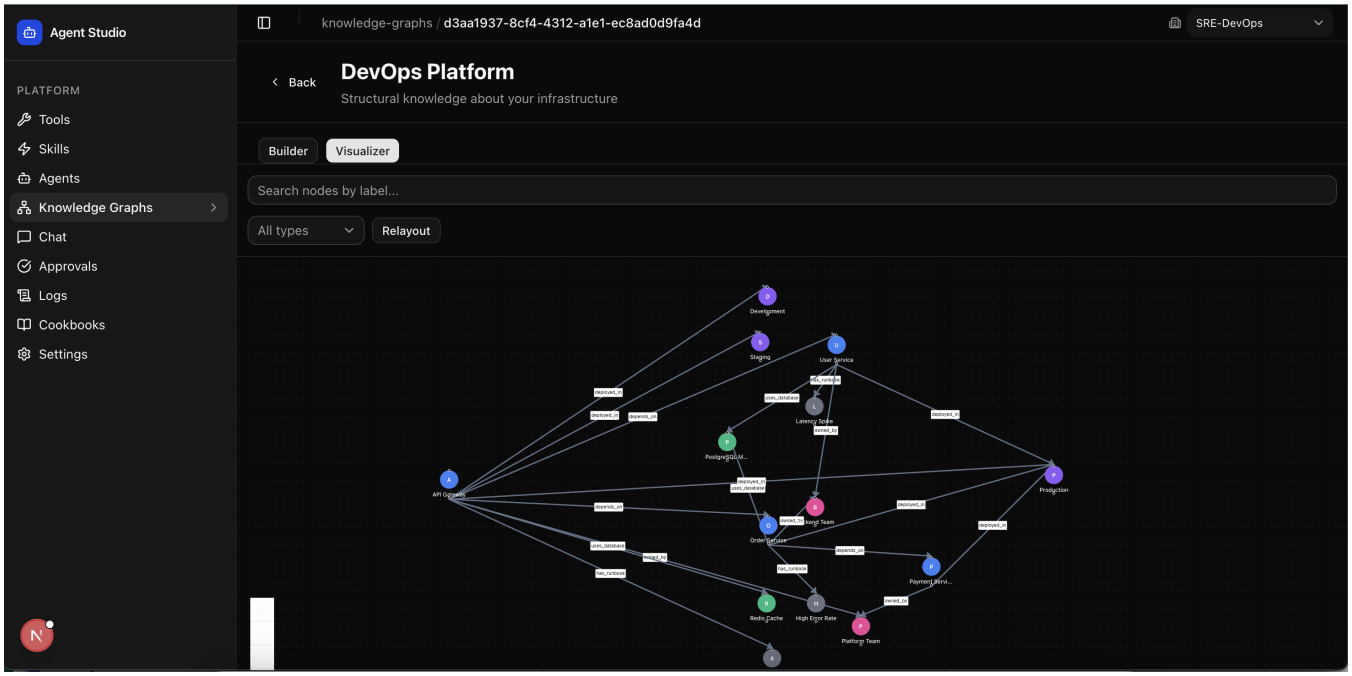
• Search for specific nodes and validate they exist in the graph

• View node properties and detailed attributes

• Explore relationships and connection paths

• Semantic search over graph content

Use Cases: Understand domain structure before building agents, validate that agent knowledge is complete, identify knowledge gaps, and plan knowledge enrichment initiatives.



3. Cookbooks

Purpose: Access and customize domain-specific agent templates for rapid deployment.

Cookbook Workflow:

• **1. Browse** available cookbooks by domain (DevOps, Security, etc.)

• **2. Review** template structure including agents, KGs, and variables needed

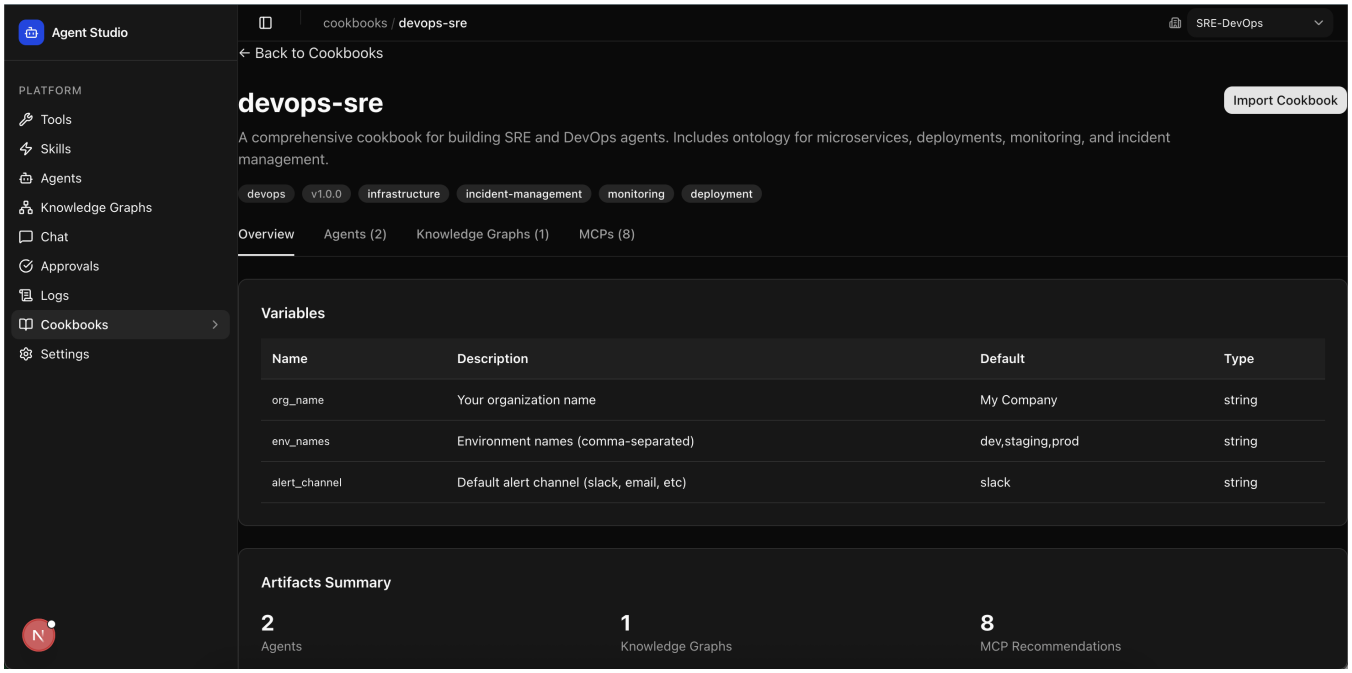
• **3. Import** cookbook into your tenant with variable customization

• **4. Customize** variables for your specific environment (org_name, env_names, etc.)

• **5. Deploy** agents and knowledge graphs automatically

• **6. Extend** with custom modifications as needed

Example: Importing DevOps-SRE Cookbook - Configure org_name to 'My Company', env_names to 'dev,staging,prod', alert_channel to '#incidents'. Result: 2 agents deployed, DevOps Platform KG created with 14 nodes and 19 edges, and 8 MCP recommendations configured.



4. Chat Interface

Purpose: Test agents in real-time conversation before deploying to production.

Chat Features:

• Send messages to agents and receive responses

• View agent reasoning and planning process in detail

• Monitor all tool calls made and their results

• Track token usage for cost estimation

• Export conversation history for documentation

• Review execution logs and error traces

Agent Response Process:

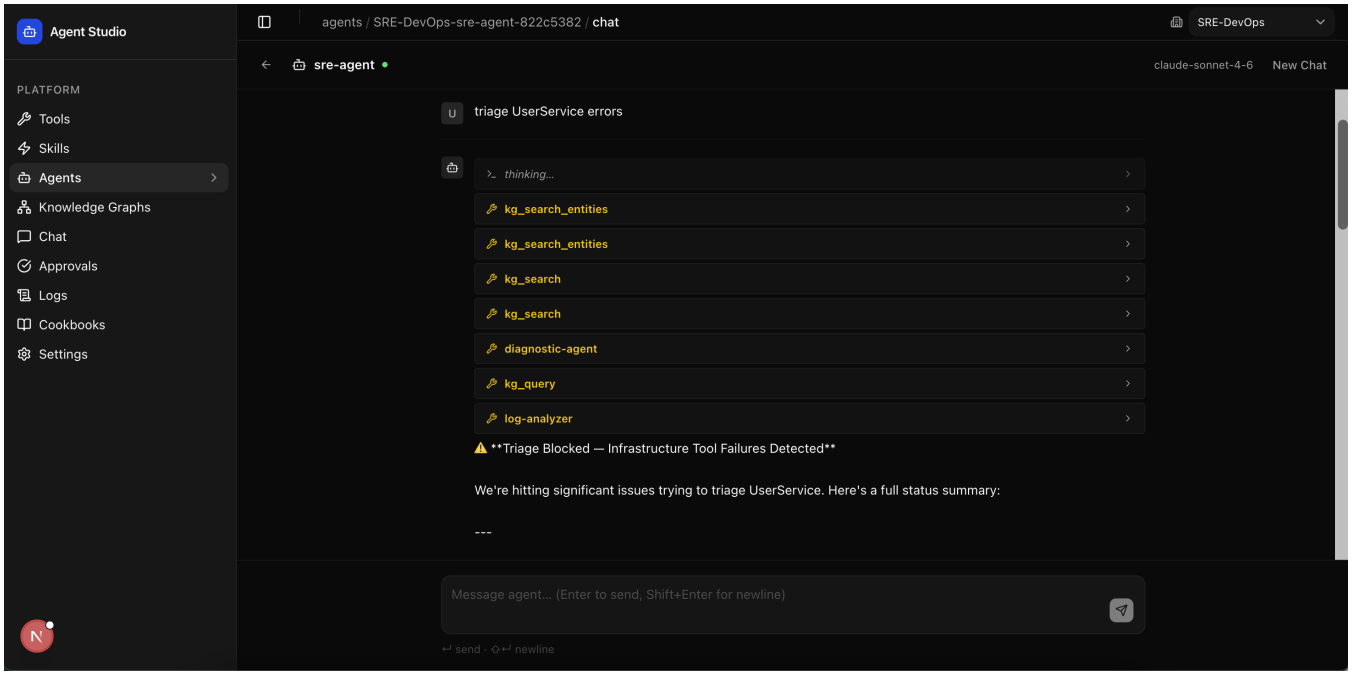
• 1. Agent receives user message

• 2. Plans approach using available tools and knowledge

• 3. Executes tools (bash, API calls, KG searches, skills)

• 4. Iterates until task complete or max iterations reached

• 5. Returns final response to user with reasoning trace



Core Concepts

Understanding these fundamental concepts is essential for operating and extending the platform effectively. Each component plays a specific role in the agent orchestration system.

1. Agents

Autonomous entities that reason about problems, invoke tools, and execute tasks. Agents are powered by LLM models and execute as Temporal durable workflows, surviving failures and maintaining an audit trail.

Agent Properties: Model (Claude/GPT), Skills (reusable capabilities), Tools (direct system access), State (memory of interactions), Constraints (max iterations, token limits)

2. Skills

Reusable agent capabilities composed from tools with domain-specific logic. Skills encapsulate tool complexity, provide versioning, enable approval workflows, and are tenant-scoped for security.

Skill Composition: Tool(s) + Configuration + Approval Rules = Skill. Example: kg-semantic-search skill uses the kg-semantic-search tool; backup-validator skill uses bash and monitoring tools.

3. Tools

Atomic functions agents invoke to accomplish work. Tools include infrastructure commands (bash), API integrations (http-request), knowledge queries (kg-search), and sandboxed code execution (code-executor).

Safety Model: Mutating tools (modify state) require approval before execution. Read-only tools (queries) execute without approval. All tool access is logged and can be revoked by administrators.

4. Knowledge Graphs

Semantic graphs storing domain ontologies and relationships. KGs enable agents to reason about complex systems, understand dependencies, and make informed decisions based on domain knowledge.

KG Structure: Nodes (typed entities like Service, Team, Infrastructure), Edges (relationships with labels like 'owns', 'depends-on'), Attributes (node properties), Schema (type system and constraints)

5. Cookbooks

Domain-specific templates for rapid agent composition. Cookbooks package complete solutions: agents, knowledge graphs, tools, and configuration variables—enabling teams to deploy production-ready agents in minutes.

Benefits: Accelerate development, enforce best practices, ensure consistency, enable knowledge sharing across teams

6. Multi-Tenancy

Secure isolation of customer data and workloads. PostgreSQL Row-Level Security (RLS) enforces tenant boundaries at the database layer—not just application code. Each agent, skill, and knowledge graph belongs to exactly one tenant.

Isolation Mechanisms: RLS at DB layer, tenant-scoped API keys, separate Temporal task queues, connection pooling per tenant, independent scaling and billing per tenant

7. HITL (Human-In-The-Loop) Approval

Critical operations require human authorization before execution. Mutating tools (bash, data modifications), high-risk workflows, and policy exceptions require approval. The approval workflow maintains audit trails and provides transparency for compliance.

Workflow: Agent plans action → Sends approval request with context → Human reviews → Approve/Reject → Agent proceeds or halts → Audit logged

Common Workflows

Workflow 1: Deploy a Domain Agent

This workflow guides you through importing a template cookbook and customizing it for your environment.

1. Browse Cookbooks: Agent Studio → Cookbooks. Select the template matching your domain (DevOps, Security, etc.)

2. Review Structure: Examine agents, knowledge graphs, and tools included. Check variables needed.

3. Import: Click 'Import Cookbook' and provide variable values (org_name='My Company', env_names='dev,staging,prod')

4. Customize: Agents are created automatically. Extend with additional skills if needed.

5. Test: Use Chat interface to send test messages and validate agent behavior.

6. Deploy: Mark agent status as 'active' and configure approval requirements.

Workflow 2: Create a Custom Agent

Build a new agent from scratch tailored to your specific needs.

1. Define Agent: Agent Studio → Agents → New. Enter name, description, and select LLM model.

2. Configure: Set max iterations to control runaway behavior, select system prompt focus.

3. Compose Skills: Add relevant skills from the catalog. Each skill provides domain-specific capabilities.

4. Test Immediately: Use Chat to send test messages and monitor tool invocations.

5. Review Logs: Examine execution traces. Identify tool failures or optimization opportunities.

6. Refine & Iterate: Adjust skill selection, refine system prompt, increase iterations if needed.

7. Activate: Set status to 'active' when satisfied with behavior.

Workflow 3: Troubleshoot Agent Failures

Systematic approach to diagnosing and fixing agent execution problems.

1. Locate Failure: Agent Studio → Logs. Find the failing execution by agent name and timestamp.

2. Analyze: Review tool calls and outputs. Identify where execution diverged from expected.

3. Check Tools: Admin Console → System Tools. Verify the tool is enabled and not blocked by approvals.

4. Validate Skills: Agent Studio → Skills. Confirm skill uses correct tool and configuration is valid.

5. Inspect Knowledge: Agent Studio → Knowledge Graphs. Search for entities agent is trying to find.

6. Fix Agent: Adjust skill selection, refine system prompt, or increase max iterations.

7. Retest: Run agent again in Chat interface. Verify fix works end-to-end.

Best Practices

Follow these guidelines to build reliable, secure, and maintainable agent systems.

Agent Design

Do:

✓ Compose agents from well-tested skills with clear purposes

✓ Use descriptive system prompts that guide behavior and focus

✓ Limit max iterations (typically 5-10) to prevent runaway loops

✓ Enable HITL approval for risky operations (bash, deployment, data changes)

✓ Test extensively in Chat before deploying to production

Don't:

✗ Give agents unlimited tool access without approval gates

✗ Mix too many unrelated skills (focus on single domain per agent)

✗ Use overly verbose system prompts that confuse intent

✗ Deploy to production without thorough testing

✗ Ignore or bypass approval requirements

Knowledge Graph Management

Do:

✓ Keep schemas consistent and well-documented

✓ Update graphs regularly to reflect current system state

✓ Validate graph integrity and consistency periodically

✓ Use semantic relationships meaningfully (not random connections)

✓ Ensure agents can discover context they need

Don't:

✗ Treat knowledge graphs as one-time snapshots

✗ Allow schema drift without documentation

✗ Create disconnected subgraphs or isolated nodes

✗ Overload nodes with unrelated attributes

✗ Forget to embed node data for semantic search

Security & Compliance

Do:

✓ Enforce tenant isolation—never bypass RLS policies

✓ Rotate MCP tokens regularly and audit access logs

✓ Enable approval for all sensitive operations

✓ Keep audit logs retained for compliance investigations

✓ Validate cross-tenant data access is technically impossible

Don't:

✗ Bypass approval workflows for convenience

✗ Mix secrets or credentials in logs or chat history

✗ Grant excessive permissions to single agents

✗ Trust unvalidated inputs from users or external APIs

✗ Delete audit logs prematurely

Monitoring & Observability

Do:

✓ Track agent execution metrics (success rates, latency, cost)

✓ Monitor tool invocation patterns for anomalies

✓ Alert on failure rate spikes or resource exhaustion

✓ Export logs regularly for analysis and archival

✓ Review cost trends to catch runaway spending

Don't:

✗ Ignore failing agents without investigation

✗ Assume everything is working (verify with dashboards)

✗ Skim monitoring dashboards without acting on alerts

✗ Let logs grow unbounded without rotation

✗ Miss early warning signs of systemic issues

Quick Reference & Troubleshooting

Service Ports

Service	Port	Description
API Gateway	8080	Entry point for all API requests
Workflow Initiator	8081	Temporal workflow dispatcher
MCP Registry	8090	MCP server hub
Agent Studio	3000	Frontend for domain architects
Admin Console	3001	Frontend for platform operators

Common Issues & Solutions

Issue	Root Cause	Solution
Agent fails with "Tool not found"	Tool not enabled in System Tools	Admin Console → System Tools → Verify tool is enabled
"Approval required" error	Mutating tool requires authorization	Review HITL settings in Skills, provide approval in Agent Studio
Knowledge graph search returns nothing	KG not imported or not embedded	Agent Studio → Knowledge Graphs → Re-import and verify nodes exist
Cookbook import fails	Missing required variables	Check all required variables provided, verify variable format
Chat timeout / max iterations exceeded	Task too complex or agent misconfigured	Increase max_iter setting, simplify task, or refocus agent

Getting Help

• **View Execution Logs:** Agent Studio → Logs. Shows detailed traces of agent reasoning and tool calls.

• **Check Audit Trail:** Admin Console → Audit Log. System-wide event history for debugging.

• **Review Architecture:** See architecture.md for system design, data flow, and integration patterns.

• **API Reference:** Admin API at port 8089 with endpoints for agents, workflows, approvals.

• **Documentation:** README.md for project overview, requirements.md for feature specifications.

Key Reminders

→ **Temporal is the only execution engine** – All agent workflows run through Temporal (no direct async/background jobs)

→ **Tenant isolation is enforced at DB layer** – PostgreSQL RLS policies prevent cross-tenant access

→ **All tool calls route through Skill Dispatcher** – Direct tool execution is not supported

→ **Approve mutating operations carefully** – Review context thoroughly before approving sensitive actions

→ **Monitor costs continuously** – LLM token usage and workflow execution minutes drive spending

→ **Keep knowledge graphs current** – Stale or incomplete KGs lead to poor agent decisions

Document Information

Generated: May 15, 2026 at 06:37 PM
Version: 1.0.0
Document: A1 Agent Engine Platform Guide - Complete How-To
For: Platform Operators & Domain Architects

This guide provides comprehensive coverage of the A1 Agent Engine platform, including Admin Console and Agent Studio interfaces, core concepts, common workflows, and best practices. For additional support, refer to the online documentation or contact your platform team.